

ORDER IDENTIFICATION

An order in an ecommerce system can have:

- Internal order ID → **ORD10001**
- Marketplace order ID → **AMZ-78945**
- ERP order ID → **ERP-55021**
- External shipment reference → **SHIP-222**
- Customer reference ID → **CUST-REF-11**

Rather than adding all these columns to **OrderHeader**, OFBiz/Moqui stores them separately in **OrderIdentification**.

Understanding each field

```
<field name="orderIdentificationTypeId" type="id" is-pk="true"/>
```

Defines **what type of ID this is**.

Examples:

orderIdentificationTypeId	Meaning
AMAZON_ORDER_ID	Amazon order number
ERP_ORDER_ID	ERP system order ID
CUSTOMER_REF	Customer reference
SHIPMENT_REF	Shipment reference

```
<field name="orderId" type="id" is-pk="true"/>
```

Links to the actual order in **OrderHeader**.

Example:

ORD10001

```
<field name="idValue" type="id-long"/>
```

Stores the actual identifier value.

Example:

AMZ-88765

```
<field name="fromDate" type="date-time" is-pk="true"/>
```

Stores when this identifier became active.

It is also part of the primary key because the same identification type may change over time.

```
<field name="thruDate" type="date-time"/>
```

Stores when the identifier stopped being valid.

If empty (`null`), it means the ID is currently active.

Relationship explanation

Relationship 1

```
<relationship type="one"
fk-name="ORDER_ID_TYPE"
related="moqui.basic Enumeration">
```

```
<key-map field-name="orderIdIdentificationTypeId"
related="enumId"/>
```

Maps identification type to enumeration values.

Example:

Enumeration table:

enumId	description
AMAZON_ORDER_ID	Amazon Order Identifier
ERP_ORDER_ID	ERP Order Identifier

Relationship 2

```
<relationship type="one"
fk-name="ORDR_ID_ORDER"
related="org.apache.ofbiz.order.order.OrderHeader">
```

```
<key-map field-name="orderId"/>
```

Links identifier back to the actual order.

Example:

OrderHeader:

orderId	customerName
ORD10001	John

Real example with data

Suppose customer John places an order.

OrderHeader:

orderId	customerName	total
ORD10001	John	2500

Now the order goes through multiple systems.

OrderIdentification table:

orderIdIdentificationType	orderId	idValue	fromDate	thruDate
AMAZON_ORDER_ID	ORD10001	AMZ-98765	2026-05-20 10:00	null
ERP_ORDER_ID	ORD10001	ERP-45678	2026-05-20 10:02	null
CUSTOMER_REF	ORD10001	REF-AB12	2026-05-20 10:05	null

So one order:

ORD10001

has three identifiers:

Amazon → AMZ-98765

ERP → ERP-45678

Customer Ref → REF-AB12

Why **fromDate** is in primary key

Imagine ERP changes the identifier later.

Old record:

Type	Order	ID	fromDate	thruDate
ERP_ORDER_ID	ORD10001	ERP-45678	Jan-1	Feb-1

New record:

Type	Order	ID	fromDate	thruDate
ERP_ORDER_ID	ORD10001	ERP-78900	Feb-1	null

You preserve history instead of overwriting.

Current active ID is found using:

WHERE thruDate IS NULL

Purpose in real OFBiz usage

This entity becomes useful when integrating with:

- Marketplace systems (Amazon, Flipkart, Shopify)
- ERP systems
- Warehouse systems
- Shipping providers
- External payment systems
- Customer-provided reference numbers

Without it:

OrderHeader

```
|— amazonOrderId
|— erpOrderId
|— shipmentReference
|— customerRef
|— paymentRef
```

The table would become messy.

Instead OFBiz does:

OrderHeader

```
|
|---- OrderIdentification
      |
      |---- Amazon ID
      |---- ERP ID
      |---- Customer Ref
      |---- Shipment Ref
```

This follows normalization and makes the system flexible for future integrations.

A Scenario Based Example

Let's take a realistic ecommerce flow because this entity becomes much easier to understand when seen in an actual business process.

Suppose you have an online company selling shoes using:

- Website for customers
- Amazon marketplace
- ERP system for inventory and finance
- Warehouse Management System (WMS)
- Shipping provider

A customer buys a shoe from Amazon.

Step 1: Customer places an order on Amazon

Customer:

Name: John

Product: Nike Shoes

Amount: ₹3000

Amazon creates its own order number:

Amazon Order ID = AMZ-78541

Now this order enters OFBiz.

OrderHeader record:

orderId	customer	total
ORD10001	John	3000

Notice OFBiz creates its own internal ID:

ORD10001

But if later someone asks:

"Find order AMZ-78541"

OFBiz only knows **ORD10001**.

So we store a mapping in **OrderIdentification**.

OrderIdentification:

orderIdIdentificationType	orderId	idValue	fromDate
AMAZON_ORDER_ID	ORD10001	AMZ-78541	20-May

Now:

AMZ-78541 ↔ ORD10001

Step 2: ERP receives the order

After order creation, OFBiz sends the order to ERP.

ERP creates:

ERP Order Number = ERP-5500

ERP uses this number internally for:

- accounting
- invoicing
- inventory allocation

Now another record gets added.

orderIdentificationTypeId	orderId	idValue
AMAZON_ORDER_ID	ORD10001	AMZ-78541
ERP_ORDER_ID	ORD10001	ERP-5500

Step 3: Warehouse receives order

Warehouse system creates a picking reference:

Pick Ticket = PICK-222

New record:

orderIdentificationTypeId	orderId	idValue
AMAZON_ORDER_ID	ORD10001	AMZ-78541
ERP_ORDER_ID	ORD10001	ERP-5500
PICK_REFERENCE	ORD10001	PICK-222

Step 4: Shipping company creates shipment

Shipping partner creates:

Tracking Number = TRK-99887

New record:

orderIdentificationTypeId	orderId	idValue
----------------------------------	----------------	----------------

AMAZON_ORDER_ID	ORD10001	AMZ-78541
ERP_ORDER_ID	ORD10001	ERP-5500
PICK_REFERENCE	ORD10001	PICK-222
TRACKING_REFERENCE	ORD10001	TRK-99887

Now imagine customer support receives this message:

"My Amazon order AMZ-78541 hasn't arrived."

The support agent searches:

```
SELECT orderId
FROM OrderIdentification
WHERE idValue='AMZ-78541'
```

Result:

ORD10001

Now OFBiz can find everything:

Order: ORD10001
ERP Number: ERP-5500
Pick Ticket: PICK-222
Tracking Number: TRK-99887
Status: Shipped

Without **OrderIdentification**, support would have no direct mapping.

Why **fromDate** and **thruDate** matter

Suppose the shipping company changes tracking numbers.

Old:

Type	Order	ID	fromDate	thruDate
TRACKING_REFERENCE	ORD10001	TRK-99887	May-20	May-21

New:

Type	Order	ID	fromDate	thruDate
TRACKING_REFERENCE	ORD10001	TRK-22211	May-21	null

You don't lose history.

You can still answer:

"What was the tracking number used yesterday?"

Visual flow

Customer Order



Amazon creates:

AMZ-78541



OFBiz creates:

ORD10001



ERP creates:

ERP-5500



Warehouse creates:

PICK-222



Shipping creates:

TRK-99887

Stored in database as:

ORD10001



+--- AMAZON_ORDER_ID → AMZ-78541



+--- ERP_ORDER_ID → ERP-5500



+--- PICK_REFERENCE → PICK-222



+--- TRACKING_REFERENCE → TRK-99887

This is why OFBiz uses **OrderIdentification**: one order travels across many systems, and each system speaks a different "ID language." This entity acts like a translation table between all of them.